Funciones almacenadas

Dónde estamos

- Hasta ahora el código vivía en un bloque anónimo: se ejecutaba y desaparecía. La función es la primera unidad que queda guardada en la base.
- Guardada significa que tiene dueño, privilegios, estado de compilación y dependencias. Todo eso empieza a importar desde hoy.

Qué vamos a hacer

- Escribir funciones que compilen a la primera y que devuelvan valor en toda ruta de ejecución.
- Decidir cuándo una función puede vivir dentro de una consulta SQL y a qué costo.
- Usar el diccionario de datos para diagnosticar, en lugar de adivinar por qué algo no compila.

Recorrido

- Sintaxis y parámetros. Invocación desde SQL y sus restricciones. DETERMINISTIC, RESULT_CACHE y PRAGMA UDF.
- Excepciones, derechos de ejecución, empaquetado y diagnóstico. Taller guiado con el script 02.

Lo que hay que tener fresco

Qué significa compilar

- CREATE OR REPLACE guarda el fuente y el código compilado. Si hay errores, el objeto queda creado pero en estado INVALID.
- Compilar no valida la lógica del negocio: valida la sintaxis y que los objetos referenciados existan y sean visibles.

Dependencias

- Si la función consulta una tabla, queda registrada una dependencia. Al alterar esa tabla, la función se marca INVALID.
- Oracle la recompila sola en la siguiente invocación. Si esa recompilación falla, el error aparece en producción, no en el despliegue.

Tipos de dato: dos mundos

- SQL conoce NUMBER, VARCHAR2, DATE, CLOB. PL/SQL además tiene BOOLEAN, RECORD y las colecciones asociativas.
- Esa diferencia es la que decide si una función puede invocarse desde una consulta o solo desde código PL/SQL.

Función, procedimiento y bloque anónimo

Criterio	Bloque anónimo	Función	Procedimiento
Se almacena en la base	No	Sí	Sí
Devuelve un valor	No	Obligatorio, uno solo	No: usa parámetros OUT
Se invoca desde SQL	No	Sí, con restricciones	No
Se reutiliza y se versiona	No	Sí	Sí
Uso natural	Probar y aprender	Calcular, formatear, validar	Registrar, actualizar, orquestar

Criterio de diseño
Si el nombre natural de la unidad empieza por un sustantivo, por ejemplo nota_definitiva, casi siempre es una función. Si empieza por un verbo de acción, por ejemplo registrar_matricula, es un procedimiento.

Anatomía de CREATE FUNCTION

```
CREATE OR REPLACE FUNCTION fn_nota_definitiva (
  p_corte1 IN NUMBER,
  p_corte2 IN NUMBER,
  p_corte3 IN NUMBER DEFAULT 0)
RETURN NUMBER            -- tipo sin precisión ni escala
IS
  c_peso_final CONSTANT NUMBER := 0.4;
  v_nota NUMBER(3,2);    -- aquí sí lleva precisión
BEGIN
  v_nota := ROUND(p_corte1*0.3 + p_corte2*0.3 + p_corte3*c_peso_final, 2);
  RETURN v_nota;
END fn_nota_definitiva;
/
```

Dos errores de sintaxis que se repiten todos los semestres

Escribir RETURN NUMBER(3,2) en la firma no compila: el tipo del retorno va sin longitud. Y la barra final no es parte del lenguaje, es la señal de fin de bloque para SQL*Plus y SQL Developer.

Parámetros: modo, valor por defecto y notación

```
-- posicional: el orden manda
v := fn_nota_definitiva(3.5, 4.0, 2.8);

-- nombrada: obligatoria si salta parámetros
v := fn_nota_definitiva(p_corte1 => 3.5,
                        p_corte2 => 4.0);
```

```
-- mixta: posicionales primero, luego nombradas
v := fn_nota_definitiva(3.5,
                        p_corte3 => 2.8,
                        p_corte2 => 4.0);
```

Regla	Por qué
En una función, use solo el modo IN	Un parámetro OUT impide invocarla desde SQL y esconde el resultado
Los parámetros con DEFAULT van al final	Si no, el llamador queda obligado a usar notación nombrada siempre
El tipo del parámetro se declara sin longitud	p_nombre IN VARCHAR2, nunca VARCHAR2(50)
Prefiera %TYPE cuando el dato viene de una tabla	La función sigue el cambio de la columna sin tocar el código

La misma función, invocada de tres maneras

```sql
-- 1. dentro de una consulta: se ejecuta una vez por fila
SELECT id_estudiante, fn_nota_definitiva(c1, c2, c3) AS definitiva
  FROM fn_nota
 WHERE fn_nota_definitiva(c1, c2, c3) >= 3;

-- 2. en código PL/SQL
v_definitiva := fn_nota_definitiva(3.5, 4.0, 2.8);

-- 3. de forma interactiva, para probar
SELECT fn_nota_definitiva(3.5, 4.0, 2.8) FROM dual;
```

El costo escondido de la primera forma

Cada invocación es un cambio de contexto entre el motor SQL y el motor PL/SQL. Además, una función en el WHERE impide usar el índice de la columna, salvo que exista un índice basado en esa misma función.

Qué debe cumplir una función para vivir dentro de SQL

Requisito	Qué ocurre si se incumple
Todos los parámetros de modo IN	La sentencia SQL no compila: PLS-00653 o error de resolución
Parámetros y retorno con tipos SQL	BOOLEAN, RECORD o una colección PL/SQL hacen fallar la consulta
Sin COMMIT, ROLLBACK ni DDL	ORA-14551: no se puede ejecutar una operación DML dentro de una consulta
Sin modificar la tabla que la consulta lee	ORA-04091, tabla mutante, el mismo problema que veremos en disparadores
Excepciones manejadas adentro	Una excepción no atrapada aborta la consulta completa, no solo esa fila

Una función que rompe alguna de estas reglas sigue siendo válida en PL/SQL: el límite es la sentencia SQL, no el lenguaje.

DETERMINISTIC: una promesa que el optimizador cree

```
CREATE OR REPLACE FUNCTION fn_limpia_nombre (p_nombre IN VARCHAR2)
RETURN VARCHAR2 DETERMINISTIC
IS
BEGIN
  RETURN UPPER(TRIM(p_nombre));
END;
/
-- habilita un índice basado en función
CREATE INDEX ix_fn_estudiante_nombre
    ON fn_estudiante (fn_limpia_nombre(nombre));
```

Qué significa prometer

Con las mismas entradas debe devolver siempre el mismo valor. Una función que lee SYSDATE, consulta una tabla que cambia o depende de NLS no es determinística: declararla así produce resultados incorrectos sin ningún error visible.

RESULT_CACHE: calcular una vez, servir muchas

```
CREATE OR REPLACE FUNCTION fn_cupo_programa (p_id IN NUMBER)
RETURN NUMBER RESULT_CACHE
IS
  v_cupo NUMBER;
BEGIN
  SELECT cupo INTO v_cupo FROM fn_programa WHERE id_programa = p_id;
  RETURN v_cupo;
END;
/
```

Cuándo ayuda	Cuándo estorba
Datos de referencia que se leen mucho y cambian poco	Tablas con DML constante: cada cambio invalida las entradas
Funciones costosas invocadas con pocos valores distintos	Dominios de entrada muy amplios: el caché se llena y se purga
Consultas repetidas por muchas sesiones a la vez	Resultados que dependen de la sesión, del NLS o de la hora

PRAGMA UDF y el costo de cruzar entre motores

Cuando la función se usa casi siempre desde SQL, se puede compilar optimizada para ese uso

```
CREATE OR REPLACE FUNCTION fn_iva (p_valor IN NUMBER)
RETURN NUMBER
IS
  PRAGMA UDF;          -- optimiza la invocación desde sentencias SQL
BEGIN
  RETURN ROUND(p_valor * 0.19, 2);
END;
/
```

Orden de preferencia ante un cálculo repetido

Primero intente expresarlo en SQL puro, que no cruza de motor. Si necesita PL/SQL y se invoca desde consultas, considere PRAGMA UDF o una expresión en línea con la cláusula WITH FUNCTION. El caché se evalúa al final.

Excepciones: toda ruta debe terminar en RETURN

```
BEGIN
  SELECT promedio INTO v_prom FROM fn_estudiante WHERE id_estudiante = p_id;
  RETURN v_prom;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN NULL;                      -- decisión explícita, no descuido
  WHEN TOO_MANY_ROWS THEN
    RAISE_APPLICATION_ERROR(-20001, 'Identificador duplicado: ' || p_id);
END;
```

ORA-06503: función sin valor de retorno

Si el flujo llega al END sin ejecutar un RETURN, el error aparece en ejecución, nunca en compilación. Ocurre casi siempre cuando el RETURN quedó dentro de un IF sin rama ELSE.

Con qué privilegios se ejecuta: AUTHID

Cláusula	Con qué privilegios corre	Dónde resuelve los nombres	Uso típico
AUTHID DEFINER (valor por omisión)	Los del dueño del objeto	En el esquema del dueño	Exponer datos de forma controlada sin dar SELECT sobre la tabla
AUTHID CURRENT_USER	Los de quien la invoca	En el esquema de la sesión que invoca	Utilitarios que deben operar sobre los objetos de cada usuario

Detalle que sorprende en auditorías
Dentro de una unidad con derechos del definidor, los privilegios recibidos a través de un rol no aplican: deben estar otorgados directamente al dueño del objeto.

La función dentro de un paquete

```
-- especificación: el contrato público
CREATE OR REPLACE PACKAGE pk_notas IS
  FUNCTION definitiva(p1 NUMBER, p2 NUMBER,
                      p3 NUMBER) RETURN NUMBER;
END pk_notas;
/
```

```
-- cuerpo: la implementación privada
CREATE OR REPLACE PACKAGE BODY pk_notas IS
  FUNCTION definitiva(p1 NUMBER, p2 NUMBER,
                      p3 NUMBER) RETURN NUMBER IS
  BEGIN RETURN ROUND(p1*.3+p2*.3+p3*.4,2); END;
END pk_notas;
```

Por qué empaquetar

Cambiar el cuerpo no invalida a quien invoca el paquete, porque la dependencia es con la especificación. Además se agrupan las unidades relacionadas, se permite la sobrecarga y se puede mantener estado durante la sesión.

Cuando no compila: dónde mirar

```
SHOW ERRORS                                    -- justo después de crear el objeto

SELECT line, position, text
  FROM user_errors
 WHERE name = 'FN_NOTA_DEFINITIVA'
 ORDER BY sequence;

SELECT object_name, status, last_ddl_time      -- estado del objeto
  FROM user_objects WHERE object_type = 'FUNCTION';

ALTER FUNCTION fn_nota_definitiva COMPILE;     -- recompilación explícita
```

El nombre se guarda en mayúsculas salvo que se haya creado entre comillas dobles: busque siempre en mayúsculas.

Errores frecuentes con funciones

Síntoma	Causa típica	Corrección
PLS-00103 en la primera línea	Tipo con longitud en la cláusula RETURN de la firma	Declarar RETURN NUMBER, sin precisión
ORA-06503	Una ruta del código termina sin ejecutar RETURN	Agregar ELSE o un RETURN final por defecto
ORA-14551	DML o COMMIT dentro de una función invocada desde una consulta	Sacar el DML, o declararla transacción autónoma si se justifica
La consulta se vuelve lenta	La función se ejecuta por fila e impide el uso del índice	Expresarlo en SQL puro o crear un índice basado en función
El objeto aparece INVALID	Cambió una tabla de la que depende	Recompilar y revisar el impacto antes de que lo haga producción

Cómo trabajar el script 02 paso a paso

Antes de ejecutar

- Abra 02_funciones_plsql19c.sql con un usuario que tenga CREATE TABLE, CREATE PROCEDURE y CREATE VIEW en su propio esquema.
- Ejecute SET SERVEROUTPUT ON. El script crea sus propias tablas con el prefijo FN_ y no depende de los otros módulos.

Durante la práctica

- Cada paso anuncia qué se va a hacer, qué debe observar y cuál es el resultado esperado. Avance paso por paso, no de corrido.
- Después de cada CREATE FUNCTION ejecute SHOW ERRORS: acostúmbrese a leer el error de compilación antes de corregir.
- Los pasos marcados como error provocado deben fallar. Compare el mensaje con la lámina de errores frecuentes.

Al terminar

- Resuelva los cinco ejercicios finales; cada uno trae pista y criterio de logrado para autoevaluarse.
- Ejecute el bloque de limpieza cuando ya no necesite las tablas de práctica.

Lo que hay que llevarse

Del criterio

- Una función calcula y devuelve. Si además modifica datos, casi siempre debería ser un procedimiento.
- El límite para usarla dentro de SQL no lo pone el lenguaje, lo pone la sentencia: solo parámetros IN y tipos SQL.
- DETERMINISTIC y RESULT_CACHE son promesas sobre el comportamiento. Si la promesa es falsa, el error no se ve, se propaga.

Del oficio

- Maneje NO_DATA_FOUND de forma explícita: es la excepción que más funciones rompe en producción.
- Empaquete: el paquete protege a quien invoca de los cambios en la implementación.

Lecturas para llegar preparado a la próxima sesión

- PL/SQL Language Reference 19c, capítulo PL/SQL Subprograms: parámetros, resolución de invocaciones y caché de resultados.
- Database Development Guide 19c, capítulo sobre uso de funciones PL/SQL en sentencias SQL.
- docs.oracle.com/en/database/oracle/oracle-database/19/lnpls